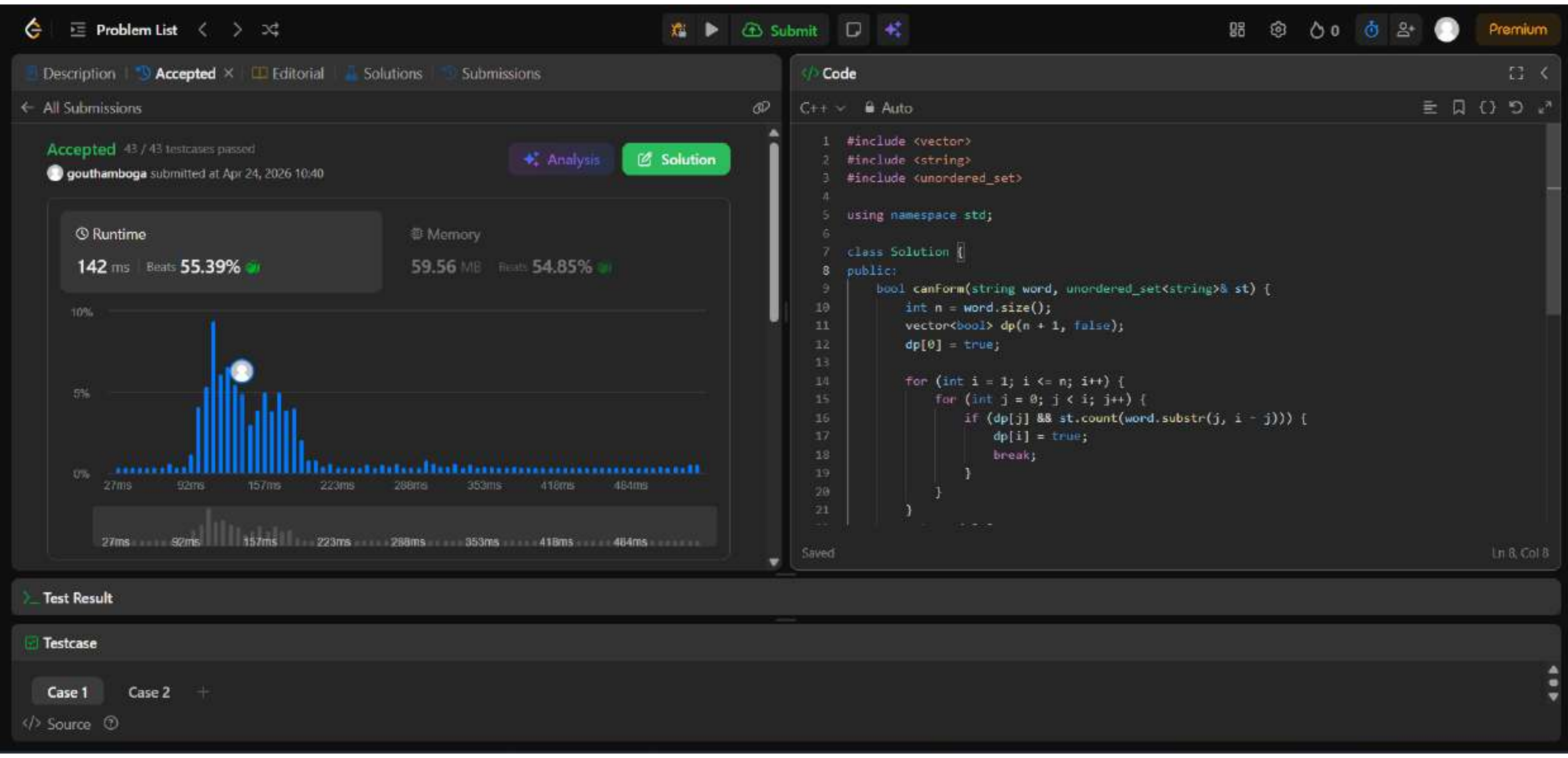
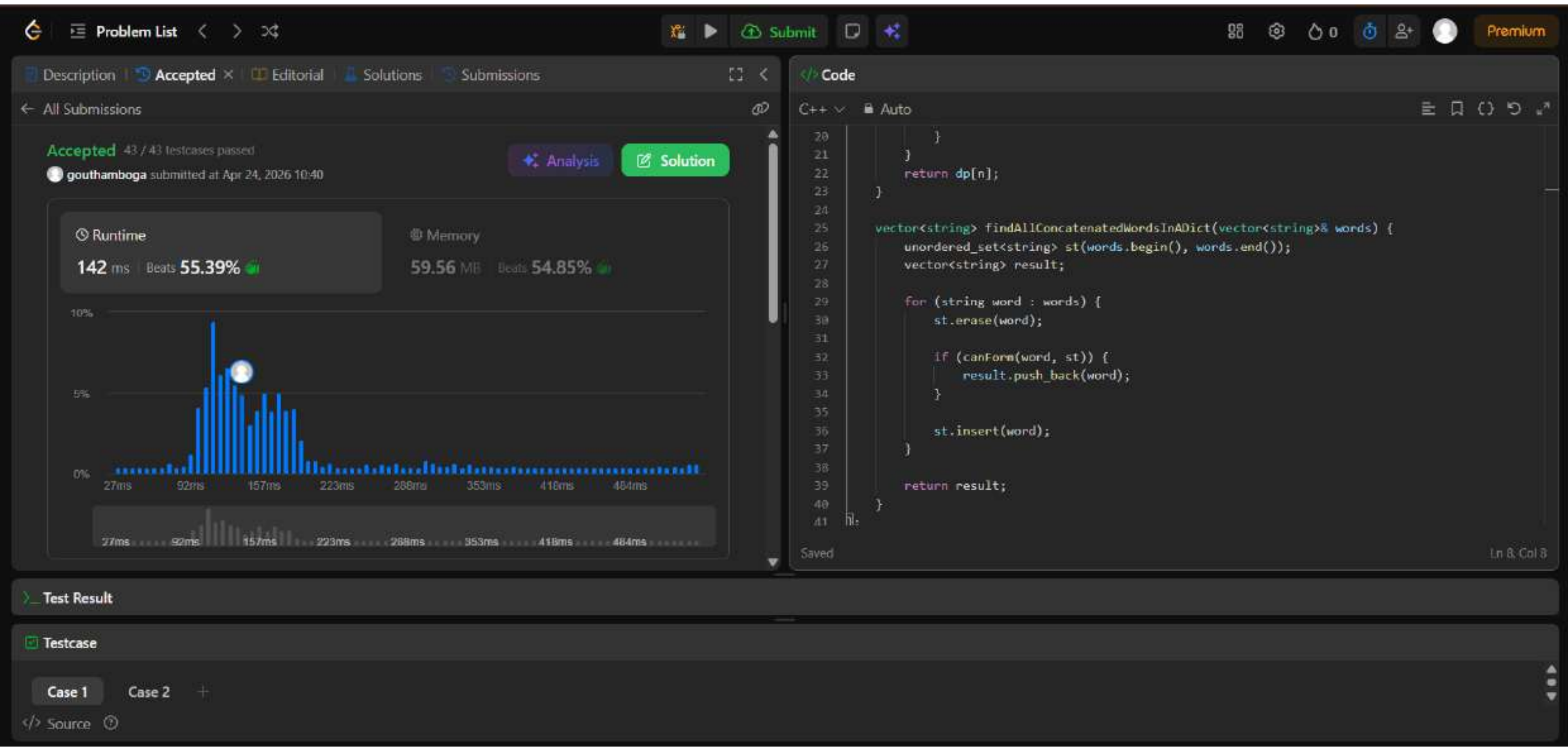


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 43 / 43 testcases passed
gouthamboga submitted at Apr 24, 2026 10:40
Analysis
Solution
Runtime
142 ms | Beats 55.39%
Memory
59.56 MB | Beats 54.85%
10%
5%
0%
27ms
92ms
157ms
223ms
288ms
353ms
418ms
484ms
Code
C++
Auto
#include <vector>
#include <string>
#include <unordered_set>
using namespace std;
class Solution {
public:
bool canForm(string word, unordered_set<string>& st) {
int n = word.size();
vector<bool> dp(n + 1, false);
dp[0] = true;
for (int i = 1; i <= n; i++) {
for (int j = 0; j < i; j++) {
if (dp[j] && st.count(word.substr(j, i - j))) {
dp[i] = true;
break;
}
}
}
Saved
Ln 8, Col 8
Test Result
Testcase
Case 1
Case 2
Source



Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 43 / 43 testcases passed
gouthamboga submitted at Apr 24, 2026 10:40
Analysis
Solution
Runtime
142 ms | Beats 55.39%
Memory
59.56 MB | Beats 54.85%
10%
5%
0%
27ms 92ms 157ms 223ms 288ms 353ms 418ms 484ms
Code
C++
Auto
}
}
return dp[n];
}
vector<string> findAllConcatenatedWordsInADict(vector<string>& words) {
unordered_set<string> st(words.begin(), words.end());
vector<string> result;
for (string word : words) {
st.erase(word);
if (canForm(word, st)) {
result.push_back(word);
}
st.insert(word);
}
return result;
}
};
Saved
Ln 8, Col 8
Test Result
Testcase
Case 1
Case 2
Source